Supervyse OPF Intel Advanced Package – RAS Offload

Revision 1.1

12/2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

Terminology 5

1 Introduction 6

2 Why RAS Offload from BIOS to BMC 7

2.1 Summary of BIOS and BMC Changes 8

3 High-Level Architecture 9

4 RAS Offload in OpenBMC 10

4.1 RasLib 12
4.2 D-bus Data Transaction 13
4.3 Debug Methods 15
4.4 Local Configuration 18

5 BIOS/BMC Handshake Flow During POST 25

6 IPMI Commands 26

6.1 RAS Capabilities (0x22) 26
6.2 RAS Send Data (0x23) 27
6.3 RAS Start(0x24) 27

7 Error Detection 28

7.1 Uncorrected Errors 28
7.2 Corrected Errors 28
7.3 ERR0 Interrupt 28
7.4 Polling 28

8 Host Signal 29

9 RAS Offload Runtime Flow 30

9.1 ERR0 Handler 31
9.2 Polling Handler 32

10 Verification 33

10.1 Test Objective 33
10.2 Hardware Setup 33
10.3 Precondition and Setups 33
10.4 BMC 37
10.5 BIOS 37
10.6 BIOS Configuration 38
10.7 Steps To Verify 41

11 Reference 46

Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2024-11-18	Nick Lin	Austin Su	Initial version
1.1	2024-12-26	Nick Lin	Austin Su	RAS Offload is added.

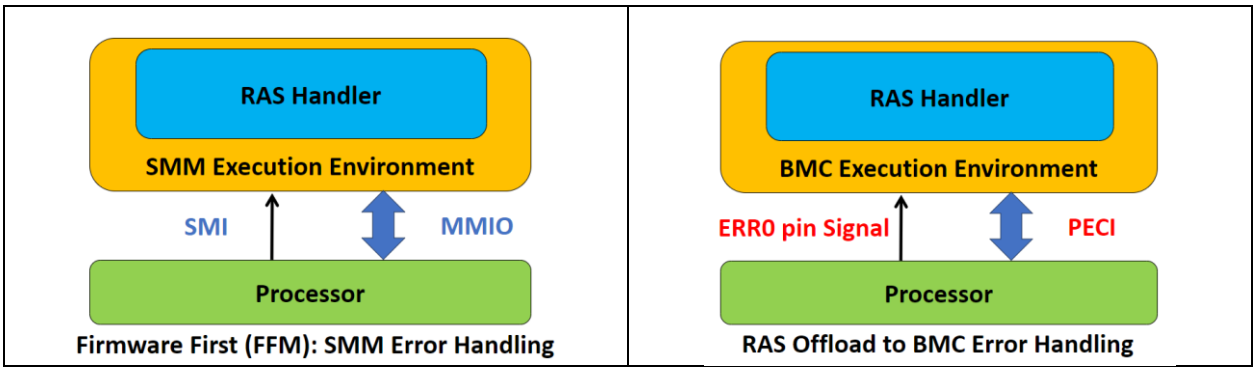
Terminology

Acronym	Definition
ASL	ACPI Source Language
asd	At-Scale-Debug
BHS	Birch Stream Platform
CEs	Correctable Errors
EGS	Eagle Stream Platform
ERR0 pin	Error Zero pin, the GPIO pin for BMC to receive CEs from CPU. For BHS platform, BMC's GPIOD0 (#24) is configured as ERR0 pin.
GNR, SRF	Granite Rapids & Sierra Forrest CPUs
MMBI	Memory Mapped BMC Interface
MMIO	Memory Mapped Input Output
MRT	Intel® Memory Resilience Technology
RAS	Reliability, Availability and Serviceability
SCI	System Control Interrupt
SMI	System Management Interrupt
SMM	System Management Mode
WHEA(log)	Windows Hardware Error Architecture Log

1 Introduction

RAS is a computer hardware engineering term involving reliability engineering, high availability, and serviceability design. Correctable Error RAS handlers execute under SMM environment. The processor and platform components generate SMI to invoke these handlers. These handlers communicate with the processor and platform components via MMIO registers in the processor address space.

RAS Offload is a technique that offloads certain error handling to BMC to minimize SMI usage. When RAS Offload is enabled, the processor and platform components assert ERR0 pin when CEs occur. The BMC monitors the state of the ERR0 pin and executes RAS handlers on signal assertion. RAS Offload feature is configured by BIOS at the boot-time, and it is plan-of-record(POR) starting BHS platform(GNR and SRF). As for EGS platform, this is plan-of-concept(POC) only.



2 Why RAS Offload from BIOS to BMC

Server platform architecture and reference implementation for many RAS features have historically relied on the broadcast SMI for event handling. In the **broadcast SMI** model, all cores enter SMI and select a monarch that handles the events. Upon completion, all cores return to the OS. The broadcast SMI model has served Intel well for many generations but is now running into multiple problems.

Performance/QoS: SMI latency increases in every generation as CPU core count increase. It becomes very complex over time and system vendors find it increasingly difficult to tune SMI handle latencies. For example, CSPs offer certain QoS guarantees to their customers, and SMI events can make it very difficult for them to keep those. e.g. 1ms duration SMI interrupt will result in 300K packets at 400G line rate.

Limits Observability: CE thresholding to reduce SMIs affects error observability.

Complexity: SMM handler complexity and race conditions scale with core count.

Security: Security experts are asking for simplifying and minimizing SMI code, due to the increase of published SMI vulnerabilities.

Therefore, that is why RAS (Only CEs) is offloaded from BIOS to BMC. The advantages are listed below.

- To speed up data communication between host and BMC.
- To help simplify the ASL code required to facilitate data transferring.

2.1 Summary of BIOS and BMC Changes

- BIOS Changes
  - BIOS Post handshake and config transfer
  - Configuration knobs (for example, Err0)
  - Disabling the corrected error handler
  - ASL RAS handler with MMBI support to get error info from BMC and notify OS via APEI/WHEA (Intel BIOS implements)
- BMC Changes
  - BMC Post handshake and config transfer
  - HAL layer to support Err0, PECI, eSPI vWire, MMBI
  - BMC requires R/W access to offloaded RAS registers over PECI
  - Runtime RAS offload code handling application
  - Porting the UEFI BIOS handlers into BMC environment (Built on top of OpenBMC - Yocto Linux based)

3 High-Level Architecture

The first-generation implementation is limited to the RAS error handling, specifically of the Correctable type. It is expected that BIOS to initialize H/W for enabled RAS features and pass required policy and data values needed by BMC RAS handlers. The term RAS handling has two potential aspects associated with it.

Error Reporting: The handler collects diagnostic information, formulates an error message, and makes it available to the OS.

Error Correcting: Actions where FW tries to recover from an erroneous or potentially erroneous scenario.

RAS handling functionality has been added to OpenBMC. host-error-monitor is intended to monitor various host error signals and take appropriate actions when triggered. e.g. logging, crashdump, and, reset, among others.

GPIO: To monitor ERR0 for error detection.

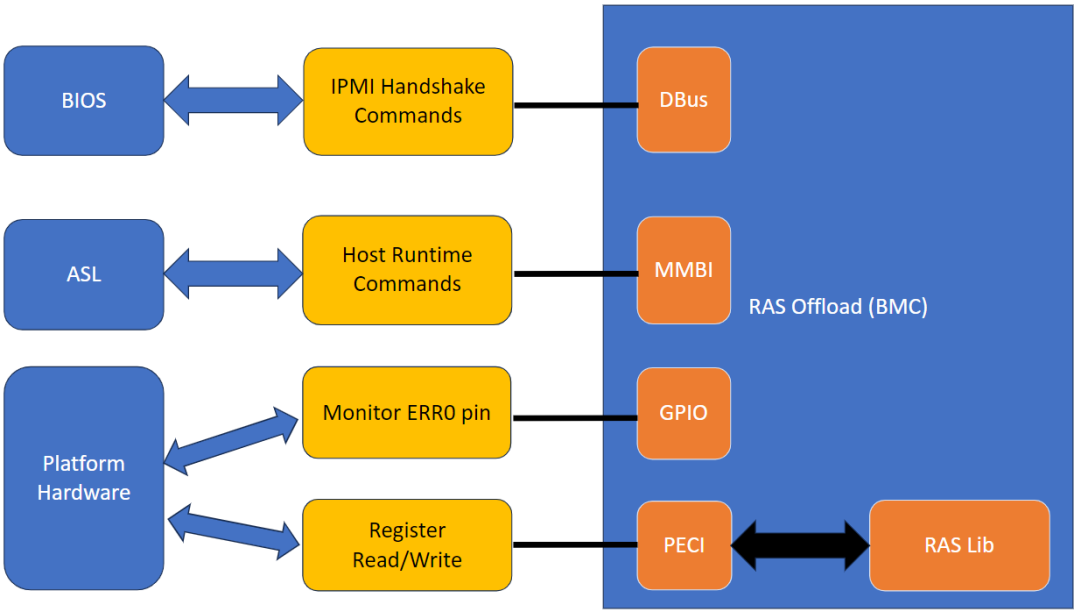
RasLib: The majority of RAS handling logic and utilizes PECI for register R/W accesses.

IPMI: The Handshake flow between BIOS and BMC. (IPMI over KCS, future MMBI)

MMBI: The data transaction of Host and BMC during run-time.

For more information, please refer to the following sections.

- BIOS/BMC Handshake Flow During POST
- RAS Offload Runtime Flow



4 RAS Offload in OpenBMC

Before building RAS Offload in OPF, the following source command must be run in your builder.

```
$ TEMPLATECONF=meta-insyde/meta-intel/meta-bhs/conf/templates/advanced/ . openbmc-env ./build/intel-bhs-insyde
```

After the environment is sourced, please issue the following command check whether RAS Offload is enabled. That is, the advanced feature *ras-offload-features* must be included in *EXTRA_IMAGE_FEATURES*.

```
$ cat <SDK>/build/intel-bhs-insyde/conf/local.conf | grep ras-offload-features
```

<SDK>/build/intel-bhs-insyde/conf/local.conf

```
# Include advanced features in the build
EXTRA_IMAGE_FEATURES:append = " \
fw-security-features \
ras-offload-features \
seamless-features \
```

The following command can be issued to build RAS Offload.

```
$ bitbake ras-manager
```

The source code hierarchy of bb recipes.

```
<SDK>
├── meta-insyde
│   └── meta-intel
│       └── meta-restricted
│           └── recipes-intel
│               └── ras-manager
│                   ├── ras-manager_%.bbappend
│                   └── ras-manager
│                       ├── 0001-fix-the-path-of-cper-log-dir.patch
│                       ├── 0002-debug-method-investigation.patch
│                       ├── 0003-get-whea-logs-ipmi-cmd.patch
│                       └── 0004-test-fake-WHEA-logs-by-ipmi.patch
└── openbmc-meta-intel
    └── meta-restricted
        └── recipes-intel
    └── ras-offload
                └── ras-manager_git.bb
```

The revision of ras-manager

```
commit 86d4d19bb6da357cc2a12e848e3edd0caada4ddd
Author: Daniel K Osawa <daniel.k.osawa@intel.com>
Date:   Mon Jul 31 13:46:45 2023 -0700

    No Size Check on ADSD Struct Verification Step (#218)

    Signed-off-by: Daniel K Osawa <Daniel.K.Osawa@intel.com>
```

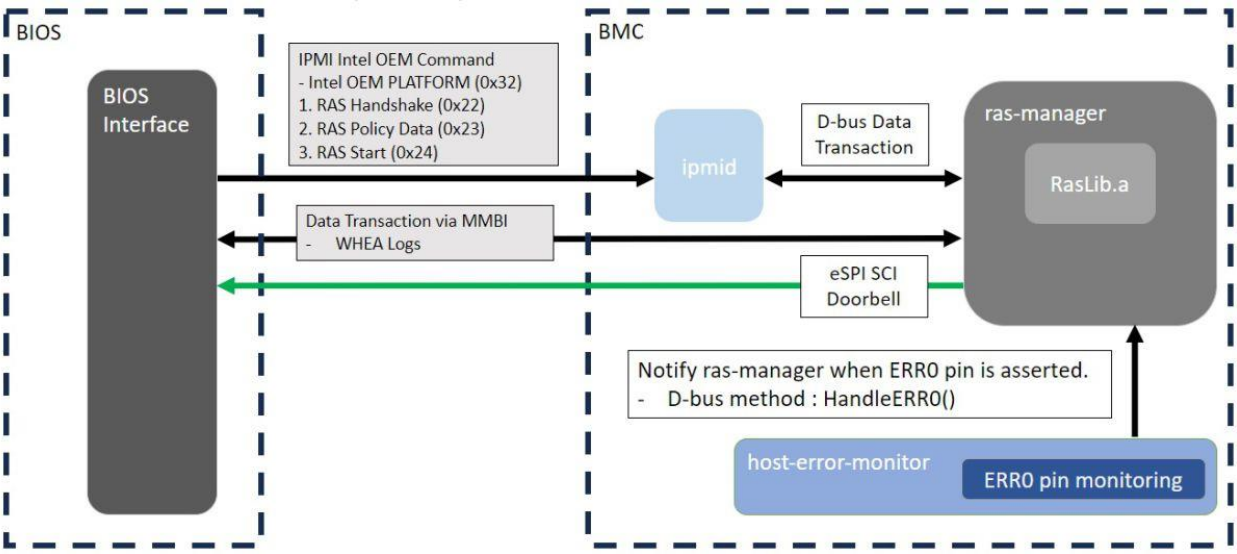
After RAS Offload is built, its source code can be found in the folder shown below.

```
<SDK>/build/intel-bhs-insyde/tmp/work/armv7ahf-vfpv4d16-openbmc-linux-gnueabi/ras-manager/0.0+gitAUTOINC+86d4d19bb6-r0
```

The source codes & patches in the folder above are listed below.

```
$ ls -al
total 316
drwxr-xr-x 20 nick.lin nick.lin  4096 十二 24 17:26 .
drwxr-xr-x  3 nick.lin nick.lin  4096 十二 24 16:58 ..
-rw-rw-r--  1 nick.lin nick.lin  1343 十一 17 10:17 0001-fix-the-path-of-cper-log-dir.patch
-rw-rw-r--  1 nick.lin nick.lin  8303 十一 17 10:17 0002-debug-method-investigation.patch
-rw-rw-r--  1 nick.lin nick.lin  9217 十一 17 10:17 0003-get-whea-logs-ipmi-cmd.patch
drwxr-xr-x  6 nick.lin nick.lin  4096 十二 24 17:26 build
-rw-r--r--  1 nick.lin nick.lin 88517 十二 24 17:26 debugsources.list
drwxr-xr-x  3 nick.lin nick.lin  4096 十二 24 17:26 deploy-rpms
drwxr-xr-x  2 nick.lin nick.lin  4096 十二 24 17:01 deploy-source-date-epoch
drwxr-xr-x 14 nick.lin nick.lin  4096 十二 24 18:26 git
drwxr-xr-x  3 nick.lin nick.lin  4096 十二 24 17:26 image
drwxr-xr-x  3 nick.lin nick.lin  4096 十二 24 17:01 license-destdir
drwxr-xr-x  3 nick.lin nick.lin  4096  八   1  2023 package
drwxr-xr-x  9 nick.lin nick.lin  4096  八   1  2023 packages-split
drwxr-xr-x  7 nick.lin nick.lin  4096  八   1  2023 pkgdata
drwxr-xr-x  7 nick.lin nick.lin  4096 十二 24 17:26 pkgdata-pdata-input
drwxr-xr-x  7 nick.lin nick.lin  4096 十二 24 17:26 pkgdata-sysroot
drwxr-xr-x  2 nick.lin nick.lin  4096 十二 24 17:26 pseudo
-rw-r--r--  1 nick.lin nick.lin 94370 十二 24 17:26 ras-manager.spec
drwxr-xr-x  6 nick.lin nick.lin  4096 十二 24 17:26 recipe-sysroot
drwxr-xr-x 10 nick.lin nick.lin  4096 十二 24 17:26 recipe-sysroot-native
drwxr-xr-x  2 nick.lin nick.lin  4096 十二 24 16:58 source-date-epoch
drwxr-xr-x  5 nick.lin nick.lin  4096 十二 24 17:26 spdx
drwxr-xr-x  4 nick.lin nick.lin  4096 十二 24 17:26 sysroot-destdir
drwxr-xr-x  2 nick.lin nick.lin 12288 十二 24 17:26 temp
-rw-r--r--  1 nick.lin nick.lin 13257 十二 24 17:01 toolchain.cmake
$
```

The software architecture is listed below. For more details about BIOS/BMC data transaction and how RAS Offload works, please refer to BIOS/BMC Handshake Flow During POST and RAS Offload Runtime Flow.



4.1 RasLib

RasLib is a C library provided by intel. It is the BMC RAS Offload FW stack heavily leverages UEFI BIOS RAS sources. Source files from the BIOS repository are selectively compiled into a single BMC library. Hardware register access APIs(MmiorRead8/16/32/etc.) are overridden in a BMC compatibility layer.

1. Initialize RAS feature
2. Execute RAS event handlers. aka error handling

Not thread safe. It is expected that the service ensures event handlers are called in a serial fashion.

4.2 D-bus Data Transaction

To show the RAS event list that BMC can support. [true is for supported, false is for not supported]

- busctl introspect com.intel.RAS /com/intel/ras
    - MemCeSupported : Memory correctable error
    - PcieCeSupported: PCIe correctable error
    - UpiCeSupported : UPI correctable error
    - SrarSupported : Currently, SRAR support is not ready yet. It is always false.

```
root@intel-obmc:/nv# busctl introspect com.intel.RAS /com/intel/ras
  NAME                                TYPE      SIGNATURE RESULT/VALUE FLAGS
  com.intel.RAS.Manager               interface -         -            -
  .CreateAdsdFile                     method    -         -            -
  .HandleERR0                         method    -         -            -
  com.intel.RAS.Manager.Handshake     interface -         -            -
  .RasSetData                         method    ay        ay           -
  .RasStart                           method    u         -            -
  .MemCeSupported                     property  b         true         emits-change
  .PcieCeSupported                    property  b         true         emits-change
  .Reserved1                          property  b         false        emits-change
  .SrarSupported                      property  b         false        emits-change
  .UpiCeSupported                     property  b         true         emits-change
  com.intel.RAS.SCIReason             interface -         -            -
  .WHEALogAvailable                   property  b         false        emits-change writable
  com.intel.RAS.WHEA                  interface -         -            -
  .GetWHEABlock                       method    y         ay           -
  org.freedesktop.DBus.Introspectable interface -         -            -
  .Introspect                         method    -         s            -
  org.freedesktop.DBus.Peer           interface -         -            -
  .GetMachineId                       method    -         s            -
  .Ping                               method    -         -            -
  org.freedesktop.DBus.Properties     interface -         -            -
  .Get                                method    ss        v            -
  .GetAll                             method    s         a{sv}        -
  .Set                                method    ssv       -            -
  .PropertiesChanged                  signal    sa{sv}as  -            -
```

To show the result of ras-offload handshaking between BMC & BIOS.

- dbus-send --system --print-reply --dest=com.intel.RAS /com/intel/ras/operational_status org.freedesktop.DBus.Properties.Get string:com.intel.RAS.OpStatus strig:enabledRasFeatures
    - EnabledRasFeatures: bitmap info
        - Bit0: PCIe correctable error.
        - Bit1: Reserved, not in use.
        - Bit2: Memory correctable error.
        - Bit3: UPI correctable error.
        - Bit4: SRAR errors.

13(0xd): PCIe, Memory and UPI correctable error are all supported.

```
root@intel-obmc:/nv# dbus-send --system --print-reply --dest=com.intel.RAS
/com/intel/ras/operational_status org.freedesktop.DBus.Properties.Get
string:com.intel.RAS.OpStatus strig:enabledRasFeatures
method return time=1680555734.405046 sender=:1.7 -> destination=:1.645 serial=30
reply_serial=2
variant       uint32 13
root@intel-obmc:/nv#
```

To get the state of Host power.

```
dbus-send --system --print-reply --dest=xyz.openbmc_project.State.OperatingSystem0
/xyz/openbmc_project/state/host0 org.freedesktop.DBus.Properties.Get
string:"xyz.openbmc_project.State.Host" string:"CurrentHostState"
```

CurrentHostState is equal to "xyz.openbmc_project.State.Host.HostState.Running", which means Host OS is boot up correctly.

```
root@intel-obmc:~# dbus-send --system --print-reply --
dest=xyz.openbmc_project.State.OperatingSystem0 /xyz/openbmc_project/state/host0
org.freedesktop.DBus.Properties.Get string:"xyz.openbmc_project.State.Host"
string:"CurrentHostState"
method return time=1679555819.211841 sender=:1.1013 -> destination=:1.1142 serial=268
reply_serial=2
variant       string "xyz.openbmc_project.State.Host.HostState.Running"
root@intel-obmc:~#
```

To fetch the state of operating system.

- dbus-send --system --print-reply --dest=xyz.openbmc_project.State.OperatingSystem0 /xyz/openbmc_project/state/os org.freedesktop.DBus.Properties.Get string:"xyz.openbmc_project.State.OperatingSystem.Status" string:"OperatingSystemState"

```
root@intel-obmc:~# dbus-send --system --print-reply --
dest=xyz.openbmc_project.State.OperatingSystem0 /xyz/openbmc_project/state/os
org.freedesktop.DBus.Properties.Get
string:"xyz.openbmc_project.State.OperatingSystem.Status" string:"OperatingSystemState"
method return time=1679627415.725213 sender=:1.1018 -> destination=:1.1112 serial=265
reply_serial=2
variant       string "Inactive"
root@intel-obmc:~#
```

4.3 Debug Methods

- After RAS handshaking, the enabled RAS offload features such as PCIe/IIO CEs and Memory CEs will be recorded in /var/lib/ras-manager/RasStart.bin
    - bit 0 - PCIe/IIO CEs
    - Bit 1 – Not defined yet
    - bit 2 - Memory CEs
    - bit 3 - UPI CEs
    - bit 4 - SRAR Errors (Not ready yet, not in use)
- To show all journal log of ras-manager.
    - journalctl –u com.intel.RAS -b –no-pager

```
root@intel-obmc:~# journalctl _PID=224
-- Journal begins at Thu 1970-01-01 00:00:08 UTC, ends at Wed 2023-05-24 06:46:39 UTC. --
May 23 09:50:30 intel-obmc ras-manager[224]: PECI set dev names to /dev/peci-default,
  /dev/peci-0
May 23 09:50:30 intel-obmc ras-manager[224]: PECI set dev name to /dev/peci-0
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_INFO::LoggerBarePrintsOnlyEnable
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_INFO::IMAGE TYPE: LOCALBIOS SRCS
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_INFO::COMMIT SHA 561c121
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_INFO::IMAGE PLATFORM GEN: SPR
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_VERBOSE::ExecutePropertyAction property ID: 5
  Action: 0x4
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_VERBOSE::Property WRITE action called on the
  MEMORY type property, PropDesc: 0x20000005, nothing to do!
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_INFO::[LoggerSetDefaultLevelSettings] start
...
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_INFO::PECI Path = /dev/peci-0
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_VERBOSE::[InitSmstGlobalStruct] start
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_VERBOSE::[InitRasGlobalDataList] start
May 23 09:50:30 intel-obmc ras-manager[224]: RAS_VERBOSE::[InitRasProperties] start
```

- Set debug level, please issue the following IPMI command.
    - For more information about the following data structure, please refer to ras.hpp.
        - *enum _RasLogLevel*
        - *enum LoggerVerbosityCommand*
        - *enum PolicyDataType_DebugSubType*
        - *enum PolicyDataType*

```
root@intel-obmc:~# ipmitool raw 0x32 0x23 0x04 0x01 0x02 0x00 <bitmap>
                                               |    |    |    |    └── enum _RasLogLevel
                                               |    |    |    └── Reserved, not in use
                                               |    |    └── 0x02: enum LoggerVerbosityCommand
                                               |    └── 0x01: enum PolicyDataType_DebugSubType
                                               └── 0x04: enum PolicyDataType
```

- To trigger ERR0 pin RAS handler, which is for simulation & debug only.
    - For more information about the following data structure, please refer to ras.hpp.
        - *enum PolicyDataType_DebugSubType*
        - *enum PolicyDataType*

```
root@intel-obmc:~# ipmitool raw 0x32 0x23 0x04 0xFF
                                               |    └── 0xFF: enum PolicyDataType_DebugSubType
                                               └── 0x04: enum PolicyDataType
```

include/ras.hpp

```
enum PolicyDataType : uint8_t
{
    MemTopology        = 0x3,
    Debug              = 0x4,
    RasPolicy          = 0x5,
    IioTopology        = 0x6,
    PrimaryPprEntries  = 0x7,
    ExtendedPprEntries = 0x8,
    ExtraDataForFeature = 0x9,
};
enum PolicyDataType_DebugSubType : uint8_t
{
    LoggerVerbosity                   = 0x1,
    DebugSetProperty                  = 0xed,
    DebugTriggerEcsHandler            = 0xee,
    DebugStats                        = 0xef,
    DebugSetUpiPollingMode            = 0xf0,
    DebugSetEcsPollingMode            = 0xf1,
    DebugSetMirrorFailoverPollingMode = 0xf2,
    DebugPlaybackRecordMode           = 0xf3,
    DebugSendDPCStatus                = 0xf4,
    DebugSetMrtMode                   = 0xf5,
    DebugPciRdReq                     = 0xf6,
    DebugMmbiPpr                      = 0xf7,
    DebugMmbiReset                    = 0xf8,
    DebugSetErr0PollingMode           = 0xf9,
    DebugSetRegMode                   = 0xfa,
    DebugTestRegs                     = 0xfb,
    DebugMmbiPcode                    = 0xfc,
    DebugMmbiWhea                     = 0xfd,
    DebugTriggerSci                   = 0xfe,
    DebugTriggerErr0RasHandler        = 0xff
};
enum LoggerVerbosityCommand : uint8_t
{
    SetDefault           = 0x1,
    SetBitmap            = 0x2,
    SetBarePrintsEnable  = 0x3,
    SetBarePrintsDisable = 0x4
};
typedef enum _RasLogLevel
{
    RAS_LOG_LEVEL_ASSERT = 0, // 0x0
    RAS_LOG_LEVEL_ERROR,      // 0x1
    RAS_LOG_LEVEL_WARN,       // 0x2
    RAS_LOG_LEVEL_INFO,       // 0x3
    RAS_LOG_LEVEL_PECI,       // 0x4
    RAS_LOG_LEVEL_BIOS,       // 0x5
    RAS_LOG_LEVEL_VERBOSE,    // 0x6

    RAS_LOG_LEVEL_INVALID,
    RAS_LOG_LEVEL_NUMBER_OF_LEVELS = RAS_LOG_LEVEL_INVALID
}RasLogLevel;
```

4.4 Local Configuration

The configuration of *ras-manager* daemon is */var/lib/ras-manager/ras-manager.json*.

ras-manager.json is generated by *ras-manager* automatically, and it can be changed on the fly.

```
root@intel-obmc:/var/lib/ras-manager# ls
ras-manager.json
root@intel-obmc:/var/lib/ras-manager# cat ras-manager.json
{
"CperLoggingEnabled": true,
"CperRecoverableLogQueueDepth":  16,
"CperRecoverableLogQueuePreserveCnt":   1,
"CperRecoverableLogDir":  "/tmp/ras-manager/output/",
"CperCorrectedLogQueueDepth":16,
"CperCorrectedLogQueuePreserveCnt": 1,
"CperCorrectedLogDir":    "/tmp/ras-manager/output/",
"CperInformationalLogQueueDepth":   16,
"CperInformationalLogQueuePreserveCnt": 1,
"CperInformationalLogDir":   "/tmp/ras-manager/output/",
"Err0PollingIntervalSeconds":0,
"UpiErrorsPollingIntervalSeconds":  60,
"Ddr5EcsPollingIntervalSeconds": 86400,
"MirrorFailOverPollingIntervalSeconds": 1,
"ImcUcnaPollingIntervalSeconds": 10,
"LoggerMsgColorCoded":   true,
"LoggerMsgVerbosity": "0xf",
"LoggerMsgFormat":1,
"AdsdFilePath":   "/tmp/address_decode.adsd",
"raslib-properties":  [{
"note":"mrt disabled=0 ; mrt enabled=1",
"name":"RAS_MRT_MODE_ID",
"id":  17,
"type":"uint8",
"val":  "0"
}, {
"note":"peci=0 ; mmbi=1 ; mixed=2",
"name":"RAS_REG_ACCESS_MODE_ID",
"id":  16,
"type":"uint8",
"val":  "0"
}, {
"note":"normal=0 ; record=1 ; playback=2",
"name":"RAS_PLAYBACK_RECORD_MODE_ID",
"id":  18,
"type":"uint8",
"val":  "0"
}, {
"note":"mmbi=0 ; tpmi=1",
"name":"RAS_B2P_MODE_ID",
"id":  25,
"type":"uint8",
"val":  "1"
}, {
"note":"see specification",
"name":"RAS_WHEA_TO_HOST_MASK_ID",
"id":  26,
"type":"uint32",
"val":  "0xFFFFFFFF"
```

```
}, {
"note": "this property controls the maximum number of retries allowed to be performed for each
  PECI transaction when PECI library returns a non-success return value or a non-success PECI
  completion code",
"name": "RAS_PECI_MAX_RETRY_CNT_ID",
"id":  27,
"type": "uint8",
"val":  "3"
}, {
"note": "mmbi retry timeout in ms",
"name": "RAS_MMBI_RESP_TIMEOUT_ID",
"id":  15,
"type": "uint32",
"val":  "40000"
}, {
"note": "wire=0 ; mctp-pcie=1 ; mctp-i3c=2",
"name": "RAS_PECI_TRANSPORT_ID",
"id":  28,
"type": "uint8",
"val":  "0"
}, {
"note": "no host=0 ; runtime smi=1",
"name": "RAS_SPPR_ENTRY_TO_HOST_MODE_ID",
"id":  29,
"type": "uint8",
"val":  "0"
}, {
"note": "dont clear=0 ; clear=1",
"name": "RAS_RRL_CLEAR_UC_ID",
"id":  30,
"type": "uint8",
"val":  "0"
}, {
"note": "try n times before giving up during CPUID socket discovery.",
"name": "RAS_CPU_ID_MAX_RETRIES_ID",
"id":  31,
"type": "uint8",
"val":  "10"
}, {
"note": "wait n second(s) between each CPUID attempt",
"name": "RAS_CPU_ID_RETRY_WAIT_INTERVAL_SEC_ID",
"id":  32,
"type": "uint8",
"val":  "1"
}, {
"note": "disabled=0 ; enabled=1",
"name": "RAS_CPER_TRACER_MODE_ID",
"id":  33,
"type": "uint8",
"val":  "1"
}, {
"note": "enable mask where each bit is associated with a scope. Scope 0 is enabled by default",
"name": "RAS_CPER_TRACER_SCOPE_MASK_0_63_ID",
"id":  34,
"type": "uint64",
"val":  "0x1"
}, {
"note": "the maximum number of CPERs per WHEA log sent to the host during PageOffline",
"name": "RAS_PAGE_OFFLINE_MAX_WHEA_ENTRY_CNT_ID",
"id":  35,
```

```
"type": "uint8",
"val":  "16"
}, {
"note": "disabled=0 (Shadow Banks access); enabled=1 (MSR McBanks access)",
"name": "RAS_MSR_MC_BANK_ACCESS_ENABLED_ID",
"id":  38,
"type": "uint8",
"val":  "0"
}, {
"note": "mask to be used during get capabilities portion of bios/bmc handshake: bit[0] pcie:
  bit[1] reserved; bit[2] memory; bit[3] upi",
"name": "RAS_CAPABILITIES_MASK_ID",
"id":  39,
"type": "uint32",
"val":  "0x0000000D"
}]
}
root@intel-obmc:/var/lib/ras-manager#
```

The Definition of key-value pairs in *ras-manager.json*

Key	Default Value	Type
CperLoggingEnabled	true	Boolean
CperRecoverableLogQueueDepth	16	Integer
CperRecoverableLogQueuePreserveCnt	1	Integer
CperRecoverableLogDir	/tmp/ras-manager/output/	String
CperCorrectedLogQueueDepth	16	Integer
CperCorrectedLogQueuePreserveCnt	1	Integer
CperCorrectedLogDir	/tmp/ras-manager/output/	String
CperInformationalLogQueueDepth	16	Integer
CperInformationalLogQueuePreserveCnt	1	Integer
CperInformationalLogDir	/tmp/ras-manager/output/	String
Err0PollingIntervalSeconds	0	Integer
UpiErrorsPollingIntervalSeconds	60	Integer
Ddr5EcsPollingIntervalSeconds	86400	Integer
MirrorFailOverPollingIntervalSeconds	1	Integer
ImcUcnaPollingIntervalSeconds	10	Integer
LoggerMsgColorCoded	true	Boolean
LoggerMsgVerbosity	“0x1”	String
LoggerMsgFormat	1	Integer
AdsdFilePath	/tmp/address_decode.adsd	String

The definition of each key in *raslib-properties*

Key	Definition
RAS_MRT_MODE_ID	MRT disabled=0; MRT enabled=1
RAS_REG_ACCESS_MODE_ID	PECI=0; MMBI=1; Mixed=2
RAS_PLAYBACK_RECORD_MODE_ID	Normal=0; Record=1; Playback=2
RAS_B2P_MODE_ID	MMBI=0; TPMI=1
RAS_WHEA_TO_HOST_MASK_ID	See specification
RAS_PECI_MAX_RETRY_CNT_ID	The maximum number of retries allowed to be performed for each PECI transaction when PECI library returns a non-success return value or a non-success PECI completion code.
RAS_MMBI_RESP_TIMEOUT_ID	MMBI retry timeout in milli second.
RAS_PECI_TRANSPORT_ID	wire=0; mctp-pcie=1; mctp-i3c=2
RAS_SPPR_ENTRY_TO_HOST_MODE_ID	No Host=0; Runtime SMI=1
RAS_RRL_CLEAR_UC_ID	D'ont Clear=0; Clear=1
RAS_CPU_ID_MAX_RETRIES_ID	Max re-try before giving up during CPUID socket discovery.
RAS_CPU_ID_RETRY_WAIT_INTERVAL_SEC_ID	The wait interval in second(s) between each CPUID attempt
RAS_CPER_TRACER_MODE_ID	Disabled=0; Enabled=1
RAS_CPER_TRACER_SCOPE_MASK_0_63_ID	The enable mask where each bit is associated with a scope. Scope 0 is enabled by default.
RAS_PAGE_OFFLINE_MAX_WHEA_ENTRY_CNT_ID	The maximum number of CPERs per WHEA log sent to the host during PageOffline
RAS_MSR_MC_BANK_ACCESS_ENABLED_ID	Disabled=0 (Shadow Banks access); Enabled=1 (MSR McBanks access)
RAS_CAPABILITIES_MASK_ID	Mask to be used during get capabilities portion of BIOS/BMC handshake. bit[0] PCIe bit[1] reserved bit[2] Memory bit[3] UPI

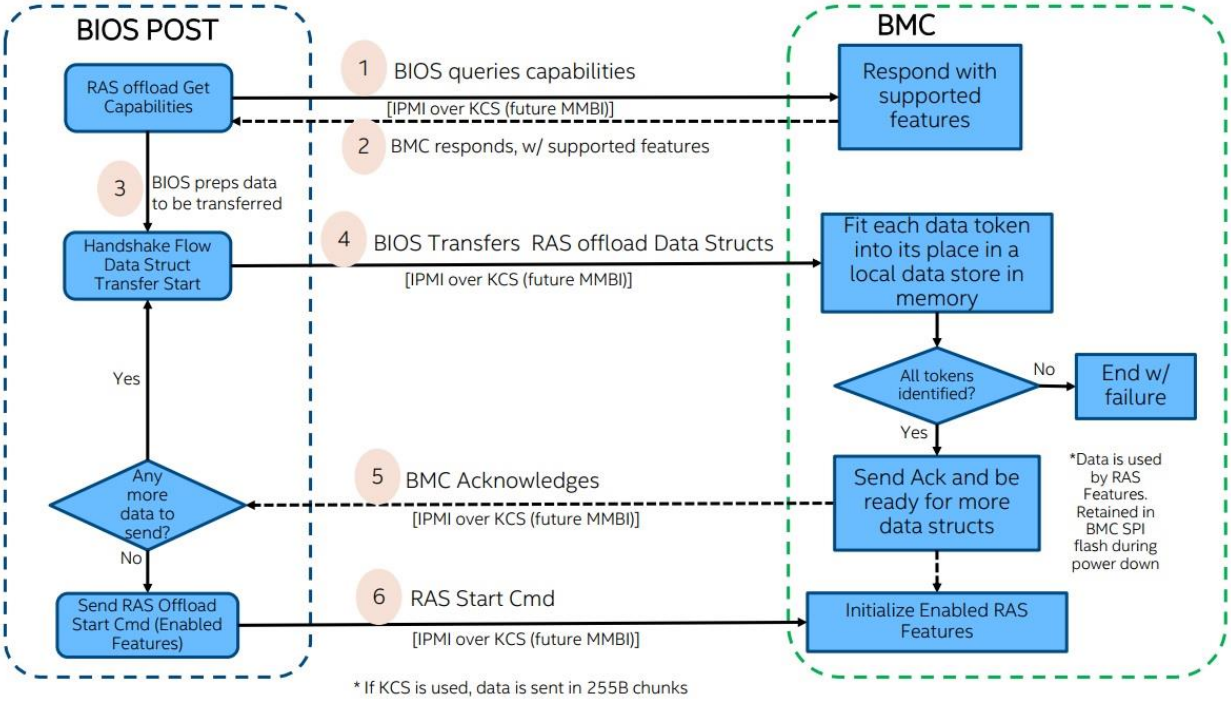
The default value and type definition of objects in *raslib-properties*

ID	Name	Default Value	Type
17	RAS_MRT_MODE_ID	0	uint8
16	RAS_REG_ACCESS_MODE_ID	0	uint8
18	RAS_PLAYBACK_RECORD_MODE_ID	0	uint8
25	RAS_B2P_MODE_ID	1	uint8
26	RAS_WHEA_TO_HOST_MASK_ID	0xFFFFFFFF	uint32
27	RAS_PECI_MAX_RETRY_CNT_ID	3	uint8
15	RAS_MMBI_RESP_TIMEOUT_ID	40000	uint32
28	RAS_PECI_TRANSPORT_ID	0	uint8
29	RAS_SPPR_ENTRY_TO_HOST_MODE_ID	0	uint8
30	RAS_RRL_CLEAR_UC_ID	0	uint8
31	RAS_CPU_ID_MAX_RETRIES_ID	10	uint8
32	RAS_CPU_ID_RETRY_WAIT_INTERVAL_SEC_ID	1	uint8
33	RAS_CPER_TRACER_MODE_ID	1	uint8
34	RAS_CPER_TRACER_SCOPE_MASK_0_63_ID	0x1	uint64
35	RAS_PAGE_OFFLINE_MAX_WHEA_ENTRY_CNT_ID	16	uint8
38	RAS_MSR_MC_BANK_ACCESS_ENABLED_ID	0	uint8
39	RAS_CAPABILITIES_MASK_ID	0x0000000D	uint32

5 BIOS/BMC Handshake Flow During POST

The BIOS/BMC handshake flow is shown.

1. BIOS queries capabilities from BMC. [IPMI over KCS (future MMBI)]
2. BMC responds with supported features.
3. BIOS prepares data to be transferred.
4. BIOS transfers "RAS offload data structures" to BMC. [IPMI over KCS (future MMBI)]
5. BMC Acknowledges. [IPMI over KCS (future MMBI)]
6. BIOS sends "RAS Start Cmd" to BMC, and then BMC initializes RAS feature.



6 IPMI Commands

Here lists the format of IPMI Commands which are in-use for the purpose of BIOS/BMC Handshaking while Host OS boot up (phase of POST).

For more details, please refer to Intel document - **734308_Intel Seamless Birch Stream Rasoffload Firmware Design Spec_Rev1_0.pdf**.

BMC-BIOS Interface IPMI Commands: Network(0x32)

Code	Description	Request Data	Response Data
0x22	RAS Capabilities	None	Byte 1: Completion Code Byte 2 to 5: Refer to Table Capabilities List
0x23	RAS Send Data	Refer to Table RAS Send Data	Byte 1: Completion Code
0x24	RAS Start	Byte 1 to 4: Refer to Table Capabilities List	Byte 1: Completion Code

6.1 RAS Capabilities (0x22)

- BIOS issues this command to query BMC for a list of RAS capabilities during the BIOS RAS policy collection phase of POST.
- This command should only be executed once during POST.
- The BMC will return back a RAS Capabilities List, and it will be the decision of BIOS which features to ultimately enable.
- BIOS will send back the enable list via the RAS Start(0x24).

Cpabilities List – Offset 0h

Bit	Feature	Description
4:31	Reserved	N/A
3	LLC/Intel® UPI Correctable Errors	0 – Not Supported, 1 – Supported
2	Memory Correctable Errors	0 – Not Supported, 1 – Supported
1	Reserved	N/A
0	PCIe/IIO Correctable Errors	0 – Not Supported, 1 – Supported

6.2 RAS Send Data (0x23)

- BIOS issues this command to send necessary data to BMC according to the RAS Capabilities List that BMC previously provided via RAS Capability Command(0x22).
- Those necessary data are Memory Topology, RAS Policies, IIO Topology, ...etc.
- This command should only be executed once during POST.

RAS Send Data (0x23)

Byte	Data Field
1	Data Type 0x1 – Reserved 0x2 – Reserved 0x3 - Memory Topology 0x4 – Debug 0x5 - RAS Policies 0x6 - IIO Topology 0x7 - Primary PPR Entries 0x8 - Extended PPR Entries
2	Data Sub Type, e.g. 0xFD, 0xFE
3 - N	Data

6.3 RAS Start(0x24)

- BIOS issues this command to tell BMC what RAS features should be enabled for RAS Offload, such as *LLC/Intel® UPI Correctable Errors*, *Memory Correctable Errors*, or *PCIe/IIO Correctable Errors*.
- This command should only be executed once during POST.

7 Error Detection

The service is responsible for determining when to handle RAS events. The first and preferred method is to trigger based on an interrupt. The second, less desirable approach is to poll for errors.

7.1 Uncorrected Errors

Error Type	RAS Offload Proposal
Catastrophic (IERR)	Offload to BMC is not supported
Fatal (MCERR)	Offload to BMC is not supported
Non-Fatal (SRAR)	Offload to BMC is not supported
Non-Fatal (UCNA/SRAO)	Offload to BMC is not supported

7.2 Corrected Errors

Error Type	RAS Offload Proposal
Core Corrected Errors (CE)	Offload to BMC is not supported
All other Uncore CEs	Offload to BMC is not supported
Local Memory CEs	SMI duties offloaded to BMC. Notify OS via SCI.
PCIe CEs	SMI duties offloaded to BMC. Notify OS via SCI.
Intel® Ultra Path Interconnect (Intel® UPI) CEs	SMI duties offloaded to BMC. Notify OS via SCI.
I3C/SPD Recovery	SMI duties offloaded to BMC.
CXL Memory CEs	SCI with optional BMC assist (CCI over MCTP)

7.3 ERR0 Interrupt

One method of an interrupt-based mechanism is to wire-OR ERR0 from each socket to a BMC GPIO signal. The BMC, of course, must configure this GPIO as input and expect it to be level triggered.

ERR0 pin in BHS platform: GPIOD0(pin#24) - input direction with active-low feature.

It is expected that ERR0 pin is de-asserted by BMC once all sources of errors have been handled. For more information about how ERR0 Interrupt takes place, please refer to ERR0 Handler.

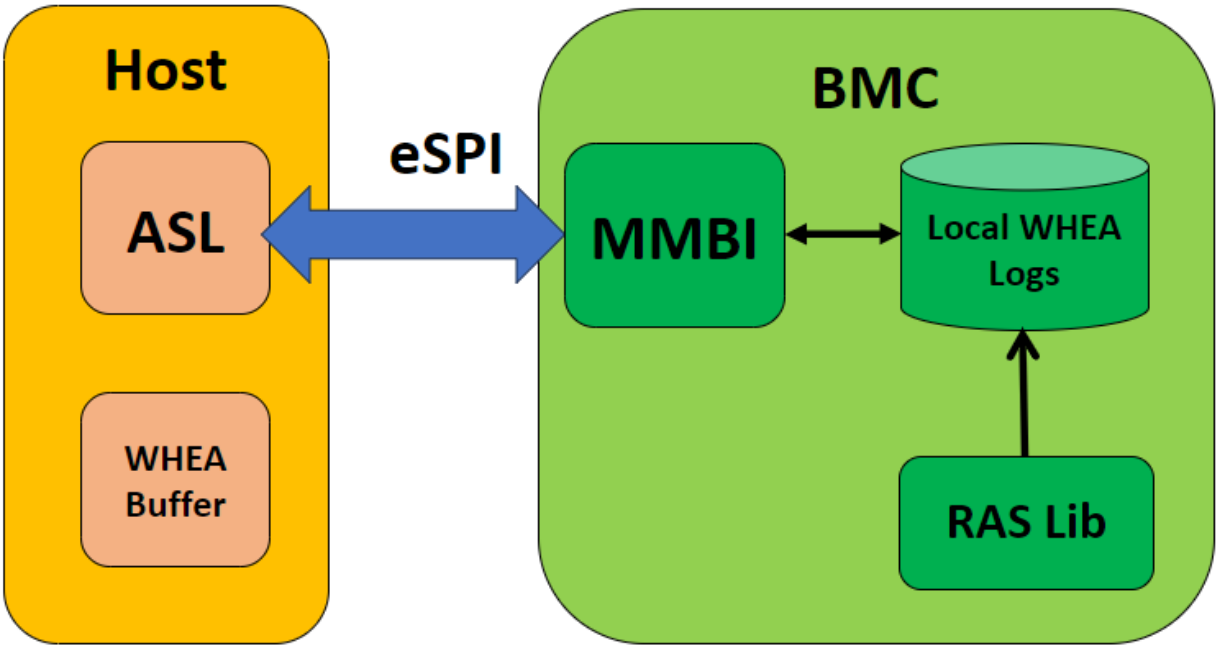
7.4 Polling

Currently, the only error source is Intel® UPI. The default polling interval will be set to one minute. The polling interval is configurable. For more info about how Polling works, please refer to Polling Handler.

8 Host Signal

After the service detects an error, it is the job of the RAS library to handle the event and construct an error message. Lastly, the BMC needs to make the OS aware that an error occurred and makes the error message accessible. This generation of RAS Offload supports the WHEA mechanism for signaling the OS of the error.

The WHEA mechanism requires that the message is copied to an ACPI-aware memory buffer that is accessible to the host. To accomplish this, the BMC utilizes **MMBI**. The MMBI specification defines a protocol that **allows packet exchanges between host software and BMC**.



9 RAS Offload Runtime Flow

There are four major tasks while CEs are discovered by BMC/BIOS.

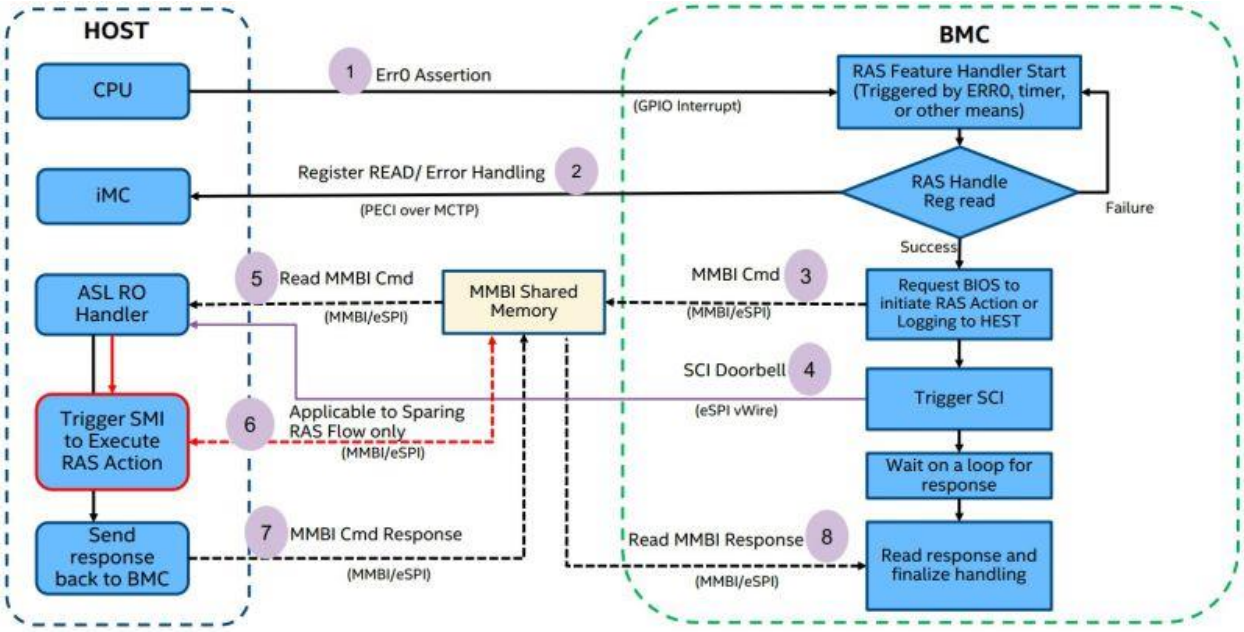
1. **Error Trigger**: CPU ERR0 pin (active low) triggers BMC to handle CEs.
2. **Error Handling**: BMC environment executed handlers communicate with the processor and platform components via PECI.
3. **OS Error Signaling**: Signal SCI to the host OS, BMC can use a BMC GPO pin connected to SCI capable GPI pin on legacy CPU or an eSPI vWire SCI# system event message to BIOS can trigger SCI to OS.
4. **Host OS Error Log Transfer**: Platform ASL code via MMBI gets error record from BMC, posts it in system memory and notifies OS via APEI/WHEA.

Below, the steps of handling CEs are introduced. **ERR0 Handler**, and **Polling Handler**.

9.1 ERR0 Handler

PCIe IIO CEs and Memory CEs are handled by ERR0 Handler. Below the flow chart of ERR0 Handler is shown.

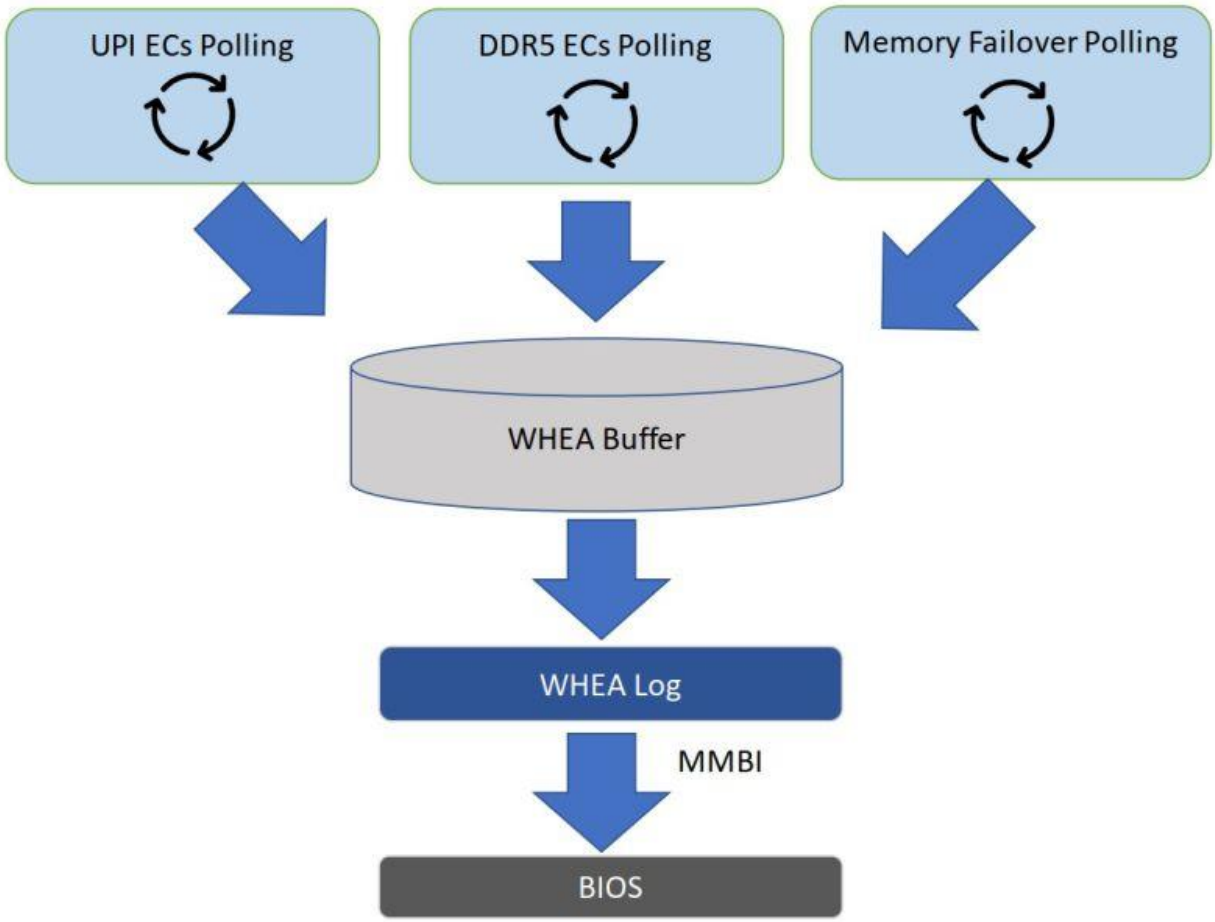
1. Once ERR0 pin is asserted, BMC will start to read global IEH, and determine which socket asserts ERR0 pin.
2. BMC invokes error handling, e.g. Register read from PECI wire.
3. WHEA logs are generated by BMC and puts it to MMBI Shared Memory.
4. BMC triggers SCI doorbell (eSPI vWire) to notify BIOS.
5. Once SCI doorbell is triggered, BIOS reads data that BMC put in the MMBI Shared Memory.
6. BIOS triggers SMI to execute RAS action. e.g. ASL RO Handler.
7. BIOS sends response message to BMC via MMBI Shared Memory.
8. The response message sent by BIOS is received by BMC and finalize handling.



9.2 Polling Handler

DDR5 ECs, Memory Failover and LLC/UPI ECs are handled by Polling Handler. The polling handler is similar to ERR0 handler but without ERR0 pin assertion. Its flow chart is shown below.

1. There are 3 threads in BMC to poll CEs via PECI wire.
2. Once CEs are detected then they are put to WHEA buffer, and then WHEA logs are generated.
3. Put WHEA logs in MMBI Shared Memory, then notify BIOS by SCI doorbell.
4. BIOS takes action to handle RAS events.
5. BIOS sends response message to BMC via MMBI Shared Memory.
6. The response message sent by BIOS is received by BMC and finalize handling.



10 Verification

10.1 Test Objective

Verify that PCIe correctable errors are offloaded to BMC, BIOS should disable SMI's and enable ERR0 pin for RAS Offload.

10.2 Hardware Setup

- CRB: Intel BHS CRB
- CPU: CPU x 1
- RAM: 16GB x 2
- PCIe Device: AVAGO MegaRAID Card

10.3 Precondition and Setups

Host OS

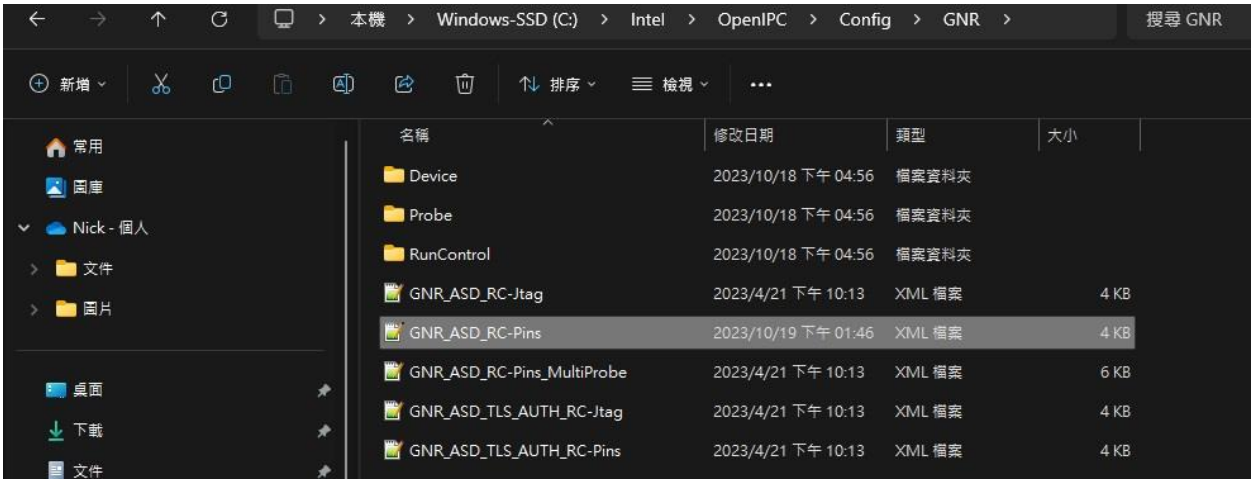
Both Windows & Linux OS can be used for verification.

- Windows
    - Edition: Windows Server 2022 Standard
    - Version: 21H2
    - OS build: 20348.2340
- Ubuntu 22.04.5 LTS

Windows PC (Where your OpenIPC is installed)

1. Please install CScripts(e.g. 686308 BHS CScripts GNR2322.4000) which provided by Intel and then set up "The BMC IP address" for OpenIPC. .e.g 192.168.22.78 in OpenIPC.

```
37
38  -->
39  <OpenIpcConfigs xmlns:xsi='http://www.w3.org/2001/XMLSchema-instance' xsi:noNamespaceSchemaLocation="../Schemas/OpenIpcC
40    <OpenIpcConfig Name="GNR_ASD_RC-Pins">
41      <PlatformControl>
42        <PublishProbeEventsAsynchronously Value="true" />
43      </PlatformControl>
44      <ProbeConfig Name="GNR_ASD_Probe">
45        <Parameters>
46          <Parameter Name="ipAddress"                    Value="192.168.22.78"/>
47          <Parameter Name="portNum"                      Value="5123"/>
48          <Parameter Name="payloadSize"                  Value="3000" />
49          <Parameter Name="maxNumBuffers"                Value="20"   />
50          <Parameter Name="securityProtocol"             Value="None"/>
51          <Parameter Name="passphrase"                   Value="tester" />
52          <Parameter Name="serverCertificate"            Value="bmc.pem" />
53          <Parameter Name="revocationFile"               Value="revocation.crl" />
54          <Parameter Name="serverCertificateVerification" Value="abort" />
55          <Parameter Name="jtagMode"                     Value="sw" />
56          <Parameter Name="chainSelectMode"              Value="single" />
57        </Parameters>
58      </ProbeConfig>
```



2. Open Windows Command Prompt with Adm priviledge, and navigate to the folder of 686308 BHS CScripts GNR2322.4000

```
cd C:\Users\nick.lin\Documents\Project\Intel\BirchStream\686308 BHS CScripts GNR2322.4000\cscripts
```

3. Key in the following command to make connection to BHS.

```
- dfmanager -> Select 0 -> Select Python38 -> dflaunch -s -> python startCscripts.py
```

```
Administrator: PowerShell
PowerShell 7.4.6
Loading personal and system profiles took 655ms.
PS C:\Users\austin.su> cd 'C:\Intel\cscript\686308 BHS CScripts GNR2344.6000\cscripts\'
PS C:\Intel\cscript\686308 BHS CScripts GNR2344.6000\cscripts> dfmanager.bat

DFPython Manager
DFHOME: C:\Intel\DFPython
Active environment: [CPython]system_py38-C:\Intel\Python38\python.exe

    0) Activate environment
    1) Add custom Python install location
    2) Clone virtual environment
    3) Create virtual environment
    4) Delete virtual environment
    q) Quit

Selection: 0

Select the environment to activate:

    0) [CPython]system_py38-C:\Intel\Python38\python.exe
    b) Back
    q) Quit

Selection: 0
Warning: This virtual environment was not generated with dfmanager

Environment successfully activated!
PS C:\Intel\cscript\686308 BHS CScripts GNR2344.6000\cscripts>
```

```
Administrator: PowerShell

DFPython Manager
DFHOME: C:\Intel\DFPython
Active environment: [CPython]system_py38-C:\Intel\Python38\python.exe

    0) Activate environment
    1) Add custom Python install location
    2) Clone virtual environment
    3) Create virtual environment
    4) Delete virtual environment
    q) Quit

Selection: 0

Select the environment to activate:

    0) [CPython]system_py38-C:\Intel\Python38\python.exe
    b) Back
    q) Quit

Selection: 0
Warning: This virtual environment was not generated with dfmanager

Environment successfully activated!
PS C:\Intel\cscript\686308 BHS CScripts GNR2344.6000\cscripts> dflaunch -s
DFHOME: C:\Intel\DFPython
Launching virtual environment ...
current_env: [CPython]system_py38-C:\Intel\Python38\python.exe
current_env_path: C:\Intel\Python38
[[CPython]system_py38-C:\Intel\Python38\python.exe] C:\Intel\cscript\686308 BHS CScripts GNR2344.6000\cscripts>
```

```
[[CPython]system_py38-C:\Intel\Python38\python.exe] C:\Intel\cscript\686308 BHS CScripts GNR2344.6000\cscripts> python startCscripts.py
Python 3.8.17 | packaged by conda-forge | (default, Jul 18 2023, 20:48:38) [MSC v.1916 64 bit (AMD64)] on win32
PythonSv running in interactive mode
Connecting to IPC API....
IPC-CLI: 4.23.23901.100, OpenIPC:5233c3ce87854ac2f44cd05 : 24.2.13416.200
Initializing IPC API....

Info - Detected relevant file modified (or added) after OpenIPC was installed: C:\Intel\OpenIPC\Config\GNR\GNR_ASD_RC-Pins.xml

Info - Detecting remote probes.
Info - OpenIPC configuration 'GNR_ASD_RC-Pins'; run control plug-in(s): 'OpenRC'; probe plug-in(s): 'ASD_JTAG'
Info - ASD plugin connected to: 192.168.22.78:5123 payloadSize: 3000 maxNumBuffers: 20 securityProtocol: None
Info - Chain 0: (Adjusted) JTAG Frequency Divisor: 14
Info - ASD I2C Bus_Select: 0 Bus_Speed: 100000
Info - ASD I2C Bus_Select: 1 Bus_Speed: 100000
Info - ASD I2C Bus_Select: 2 Bus_Speed: 100000
Info - ASD I2C Bus_Select: 3 Bus_Speed: 100000
Info - Hardware GPIO config = 0
Info - JTAG Mode = SOFTWARE
Info - JTAG Chain Select Mode = SINGLE
Info - ASD Probe plugin version: v1.5.8_ASD_BMC_v1.5.1_<35.24.44.1237>
Info - Added debug port 0 for probe ASD_JTAGProbe
Info - Detected IO_V1_CLTAP A0 on JTAG chain 0 at position 0

Info - Survivability Preset was requested, but no supported devices were detected.  Preset will be applied when a supported device is detected.
TargetEvent: Power : On                 -- 16:24:50.587243 2024-12-24
TargetEvent: PowerDomain : PCH : On     -- 16:24:50.588264 2024-12-24
TargetEvent: PowerDomain : CPU : On     -- 16:24:50.589257 2024-12-24
Target System: GNR - GraniteRapids
```

10.4 BMC

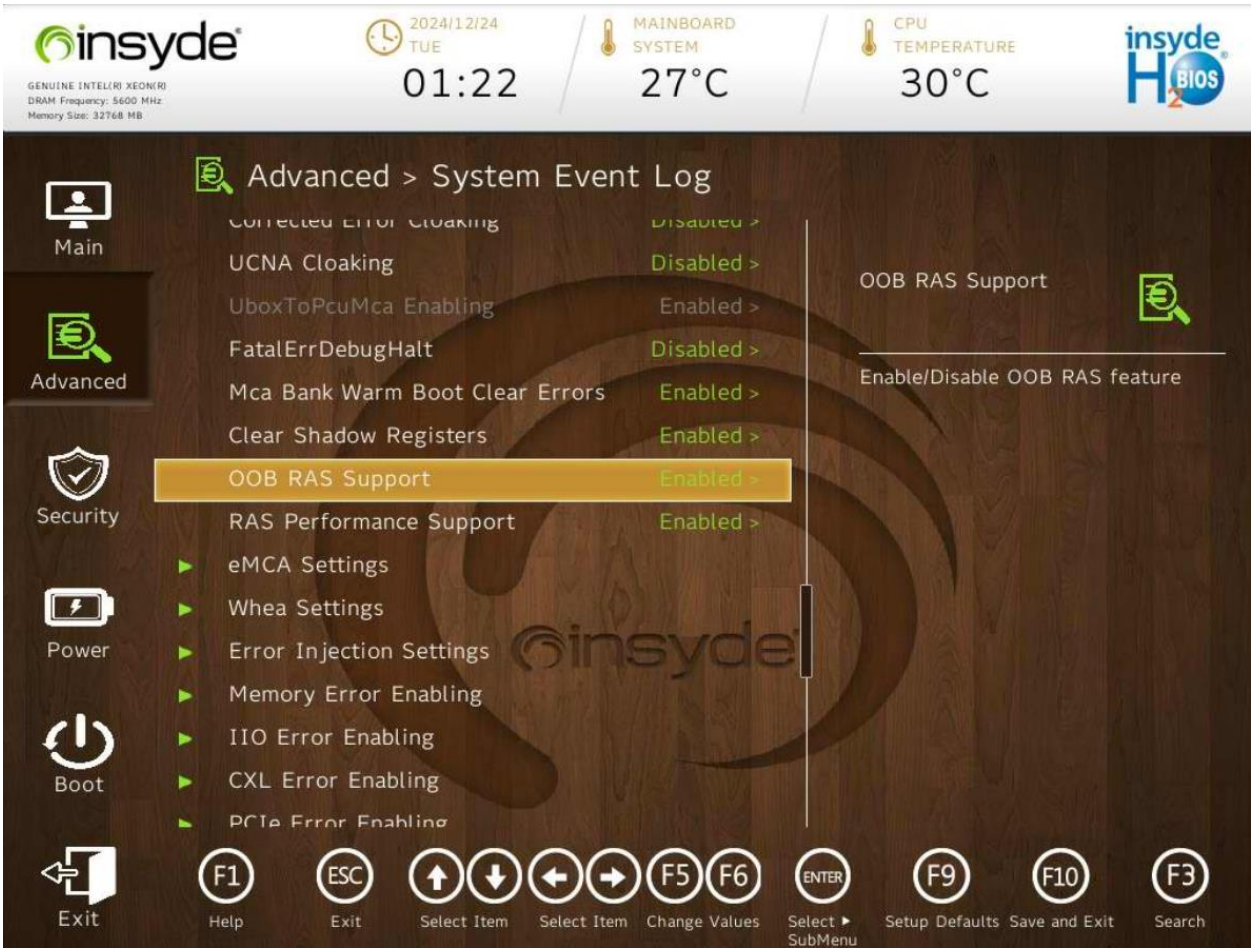
In BMC, **asd** service is used to handle error injection from your PC via Internet, which must be activated during the verification. If **asd** service is not activated, please issue "asd -u &" to let asd service run in plain TCP, no SSL.

```
root@bmc-mac6eb3fad1ebc6:~# asd -u &
[1] 2521
root@bmc-mac6eb3fad1ebc6:~#
```

10.5 BIOS

Please consult Insyde BIOS team for more information. Below is the version of Insyde BIOS we use to verify RAS Offload for your reference.

10.6 BIOS Configuration



Customer Scripts (CScripts) Enabling

The following is a quick reference of the key BIOS knobs for CScripts:

- Advanced > Socket Configuration > Processor Configuration > Lock Chipset = Disable
- Advanced > Socket Configuration > Processor Configuration > MSR Lock Control = Disable
- Advanced > Socket Configuration > IIO Configuration > DFX Global Configuration >
    - EV DFX Features = Enable
    - DIsable BIOS Done = [X]

Error Reporting Enabling

- Advanced > System Event Log > WHEA Settings >
    - WHEA Support = Enable
    - WHEA Log Memory Error = Enable
    - WHEA Log Processor Error = Enable
    - WHEA Log PCI Error = Enable

Error Injection Enabling

- Advanced > System Event Log >
    - System Errors = Enable
    - System Memory Poison = Enable
- Advanced > System Event Log > Error Injection Settings >
    - Mca Bank Error Injection Support = Enable
    - WHEA Error Injection Support = Enable
    - Whea PCIe & CXL Error Injection Support = Enable
    - Whea PCIe & CXL Error Injection Action Table = Enable
- Advanced > System Event Log > IIO Error Enabling >
    - IIO/IBL Global Error Support = Enable
    - IIO MCA Support = Disable
    - IIO eDPC Support = Disable
- Advanced > System Event Log > PCIe Error Enabling >
    - PCIE Corrected Error Threshold Counter = Disable
- Advanced > System Event Log > eMCA Settings > LMCE Support = Enable
- Advanced > System Event Log > Memory Error Enabling > Memory Corrected Error = Enable
- Advanced > Socket Configuration > Uncore Configuration > Uncore General Configuration > Directory Mode Enable = Disable

Firmware First Model

- Advanced > System Event Log > eMCA Settings >
    - EMCA Logging Support = Enable
    - EMCA CMCI-SMI Morphing = EMCA gen2 CSMI
    - EMCA CMCI-SMI Threshold = 0
    - EMCA MCE-SMI Enable = EMCA gen 2 - MSMI
- Advanced > System Event Log > IIO Error Enabling > Os Native AER Support = Disable

Crash Log

- Advanced > System Event Log > Crash Log Enabling > CPU CrashLog Feature = Disable

RAS Offload

- Receiver Error
    - Advanced > System Event Log > OOB RAS Support = Enable
- CTO Triggers DPC
    - Advanced > System Event Log > OOB RAS Support = Enable
    - Advanced > System Event Log > IIO Error Enabling > IIO eDPC Support = On Fatal and Non-Fatal Errors
- Intel UPI CE (Polling)
    - Advanced > System Event Log > OOB RAS Support = Enable
    - Advanced > System Event Log > eMCA Settings > EMCA CMCI-SMI Morphing = Disable

10.7 Steps To Verify

1. Confirm no error is seen in the root port and corresponding IEH hierarchy, please type the following CScripts command.

```
>>> halt()
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.errcorsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rooterrsts
0x0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtsts
0x0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtctl.show()
```

```
Administrator: GNR_CScripts
WARNING!!! ltssm_gnr should not be imported directly. Import pysvtools.pciedebug.l
tssm instead
Socket 0: detected Server-SP, 44 cores, A2, PCI Segment 0, Bus List = [0x00, 0x01,
 0x03, 0x05, 0x06, 0x08, 0x0a, 0x3b, 0x6b, 0x9b, 0xcb, 0xfe, 0xff]
>>> halt()
>>>
    [GPC core group] Halt on 48 devices -- 16:26:32.840575 2024-12-24
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.errcorsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rooterrsts
0x0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtsts
0x0
>>> sv.socket0.io0.uncore.globalieh.iehregs.gsysevtctl.show()
0x00000000 : reserved (31:03) -- Reserved
0x00000001 : fa_sysev_m (02:02) -- Fatal Error Mask(FA_SYSEV_M) Fatal Error System
 Event Mask: 1 - Unmask system event reporting of the fata...
0x00000001 : nf_sysev_m (01:01) -- Nonfatal Error Mask(NF_SYSEV_M) NonFatal Error
System Event Mask: 1 - Unmask system event reporting of th...
```

2. Execure error injection. Inject a PCIE CE error to PCIe root port by CScripts command.

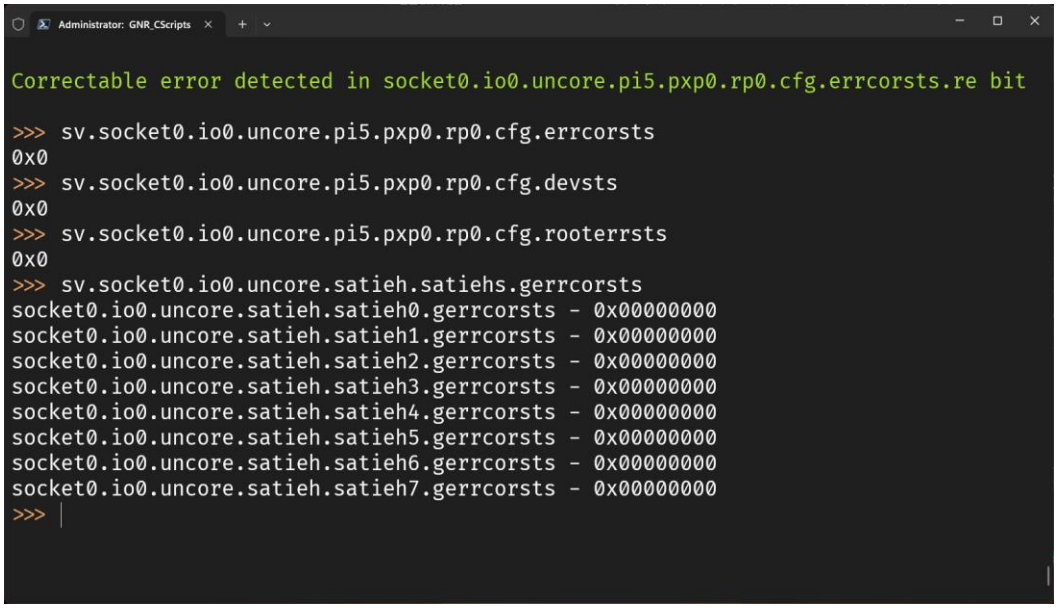
```
>>> ei.utils.resetInjectorLockCheck()
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp0", errType="ce")
```

```
Administrator: GNR_CScripts

>>> ei.utils.resetInjectorLockCheck()
Initiating an SMI

    [GPC core group] Resuming on 48 devices -- 16:27:41.830408 2024-12-24
Injectors LOCKED on GNR0_C0_T0
Injectors LOCKED on GNR0_C0_T1
Injectors LOCKED on GNR0_C1_T0
Injectors LOCKED on GNR0_C1_T1
Injectors LOCKED on GNR0_C2_T0
Injectors LOCKED on GNR0_C2_T1
Injectors LOCKED on GNR0_C3_T0

    [GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 16:27:47.786769 202
4-12-24
    [GNR0_C0_T1] SMM entry Break at [0x10:0000000000008000] -- 16:27:47.787775 202
4-12-24
    [GNR0_C1_T0] SMM entry Break at [0x10:0000000000008000] -- 16:27:47.788773 202
4-12-24
    [GNR0_C1_T1] SMM entry Break at [0x10:0000000000008000] -- 16:27:47.789774 202
4-12-24
```

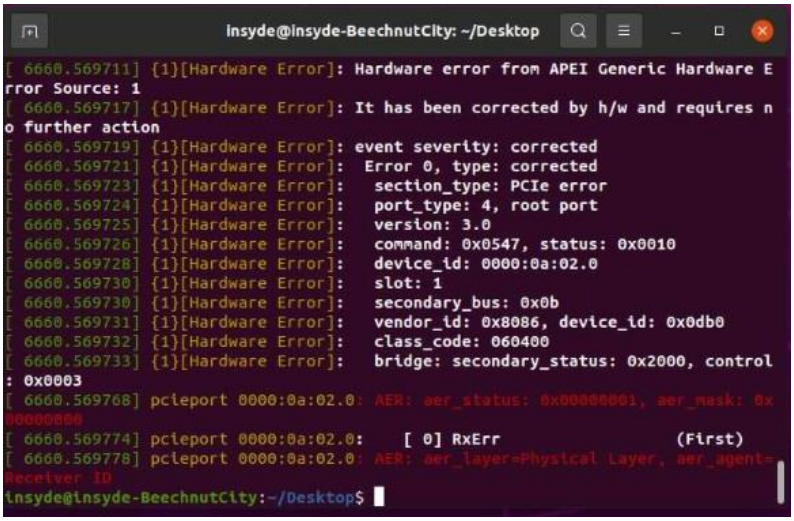
```
Administrator: GNR_CScripts
>>> ei.iio.injectPcieError(socket=0, port="pxp0.rp0", errType="ce")
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!
Initiating an SMI

    [GPC core group] Resuming on 48 devices -- 16:29:07.129008 2024-12-24
Injectors UNLOCKED on GNR0_C0_T0
Injectors UNLOCKED on GNR0_C0_T1
Injectors UNLOCKED on GNR0_C1_T0
Injectors UNLOCKED on GNR0_C1_T1
Injectors UNLOCKED on GNR0_C2_T0
Injectors UNLOCKED on GNR0_C2_T1

    [GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 16:29:12.977535 202
4-12-24
    [GNR0_C0_T1] SMM entry Break at [0x10:0000000000008000] -- 16:29:12.979488 202
4-12-24
    [GNR0_C1_T0] SMM entry Break at [0x10:0000000000008000] -- 16:29:12.980546 202
4-12-24
    [GNR0_C1_T1] SMM entry Break at [0x10:0000000000008000] -- 16:29:12.980546 202
4-12-24
```

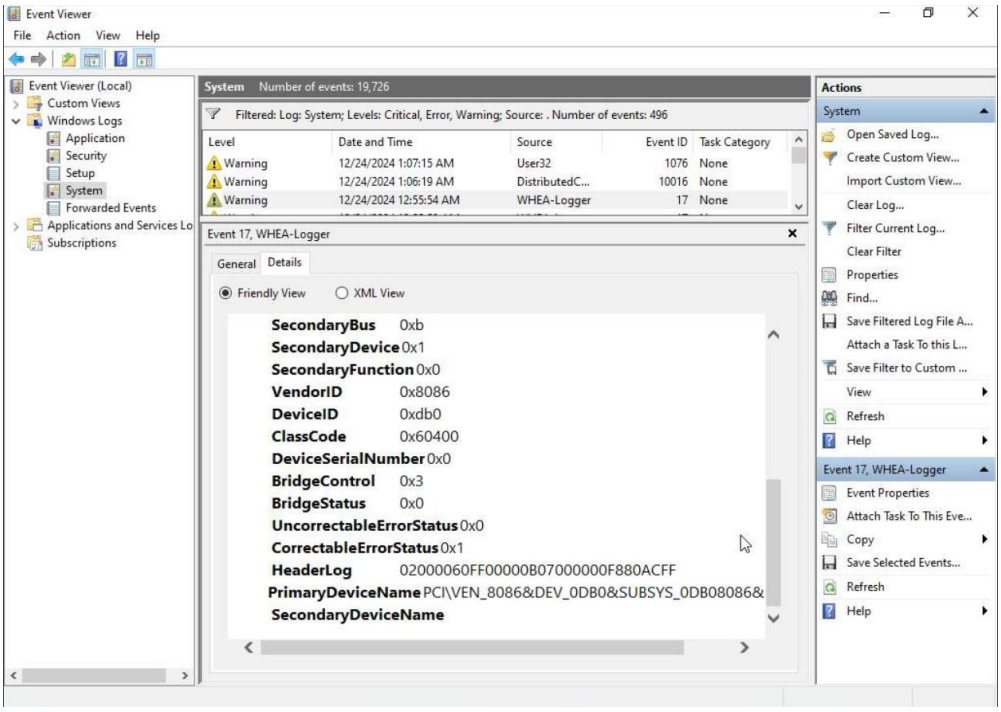
3. ERR0 pin is assert and BMC RAS Offload will be activated. Ensure logging registers are cleared.

```
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.errcorsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.devsts
0x0
>>> sv.socket0.io0.uncore.pi5.pxp0.rp0.cfg.rooterrsts
0x0
>>> sv.socket0.io0.uncore.satieh.satiehs.gerrcorsts
```



4. For Host with ubuntu installed, check kernel logs by issuing "dmesg" command in Host PC. And we can see lots of hardware error messages, which are sent by BMC via MMBI Shared Memory. For Host with Windows installed, please navigate to Event Filter, and the same hardware error messages as ubuntu OS are shown.





5. Issue the following command to check RAS manager logs in BMC console. Then search for the keyword “ERR0 Asserted” and “PCIe Error Event: Severity: ...”.

```
journalctl -u com.intel.RAS -b --no-pager
```

As shown below, the PCIe CEs are handled by ras-manager daemon correctly.

```
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_WARN::IIO_TOPOLOGY Socket[0].Stack[15].Bus value: 0xff does not match CPUBUSNO value: 0x00
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] Arming ECS Poller (expiry:86400s)...
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] Arming Mirror Failover Poller (expiry:1s)...
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] Arming IMC UCNA Poller (expiry:10s)...
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] Dumping RAS Feature Init Summary...
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] *IEH Correctable Handling: Enabled
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] *IMC Correctable Handling: Enabled
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager]   *ECS Poller: Running
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager]   *Mirror Failover Poller: Running
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager]   *IMC UCNA Poller: Running
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] *UPI Correctable Handling: Enabled
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager]   *UPI Poller: Running
Dec 24 08:20:37 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] *SRAR Errors Handling: Disabled
Dec 24 08:20:40 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] The current Host Boot State set to the EndOfPost
Dec 24 08:29:18 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] ERR0 Asserted
Dec 24 08:29:18 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] call RAS handler - RAS_IEH_HANDLE_CE_API()
Dec 24 08:29:18 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[event_listener] PCIe Error Event: Severity: CORRECTED, Slot 1 Seg 00h, PrimBus 0ah, SecBus 0bh, Dev 02h, Fun 00h, Vid 8086h
, Did 0db0h
Dec 24 08:29:18 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] The Memory Failure Prediction support is DISABLED.
Dec 24 08:29:18 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[whea_listener] PCIe Error CPER added to WHEA buffer.
Dec 24 08:29:18 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] call RAS handler - RAS_IMC_HANDLE_CE_API()
Dec 24 08:29:18 bmc-mac0007e9345b24 ras-manager[375]: RAS_INFO::[ras_manager] RAS_GET_WHEA_LOG_API (st, wheaLogSzBytes) = (0x0, 0x12C)
```

11 Reference

- 724564 DataCenterCloud DCDC 2022 RAS OFFLOAD R1.pdf
- 742874 Birch Stream Platform RAS Technology Integration and Validation Guide_Rev0.3
- 734308 intel-seamless-birch-stream-rasoffload-fwdesign-spec-r0p7.pdf
- 633972 IntelSeamlessUpdate RAS Offload Spec Rev0 9.pdf
- 634865 Memory-Mapped BMC Interface MMBI Rev0 7.pdf